

JavaScript Execution Context & The Call Stack

A Comprehensive Technical Reference Guide

In JavaScript, the **Execution Context** is the conceptual environment created by the JavaScript engine to evaluate and execute code. Every piece of running code executes within a specific context that handles scoping, variable registration, and the binding of the `this` keyword.

The Three Types of Execution Contexts

1. Global (GEC)

The base/default environment. Created when the script first loads.

- Only **one** GEC exists per program.
- Creates the global object (`window` or `global`).
- Binds `this` to the global object.

2. Functional (FEC)

Created dynamically whenever a function is actively **invoked**.

- Each function call generates a brand-new FEC.
- Scopes are local; parameters and arguments are registered here.
- Can access parent scopes but protects inner data.

3. Eval Context

Generated by code executed inside an `eval()` function block.

- Rarely used due to severe security vulnerabilities.
- Introduces performance overheads; modern conventions completely avoid it.

Lifecycle Flowchart: Call Stack Management

JavaScript uses a single-threaded **Call Stack** to manage execution contexts. Follow the linear progression below to visualize how contexts are pushed, evaluated, and popped off synchronously.

1. Script Loads → Global Execution Context (GEC) Created
Allocates memory; pushes to bottom of Call Stack



2. Invocation: firstFunction() is Called
New Functional Context (FEC) pushed to top of Stack



3. Nested Call: secondFunction() is Called
Active execution shifts to secondFunction FEC (Top)



4. Completion: secondFunction() Finishes
FEC popped off stack & garbage collected; returns to firstFunction



5. Script Completes → GEC Remains Idle
Call Stack returns to clean, baseline Global state

Code Execution Architecture Walkthrough

The following example tracks variable declarations and deep scope manipulation across alternating context barriers.

```
// 1. Engine boots up in the Global Execution Context (GEC)
let globalVar = "I am Global";

function firstFunction() {
  // 3. Brand new Functional Context created for firstFunction()
  let firstVar = "Inside First";
  console.log(globalVar); // Successfully resolves outward to GEC variable

  secondFunction(); // Invoking nested routine
  // 6. Control shifts backward to this state after secondFunction resolves
}

function secondFunction() {
  // 4. Distinct Functional Context isolated for secondFunction()
  let secondVar = "Inside Second";
  console.log(secondVar);
  // 5. execution finishes: context is popped and totally purged
}

firstFunction(); // 2. Triggering call: pushes firstFunction to the stack
```

Step-by-Step State Matrix

Step	Call Stack Allocation State	Under-the-Hood Engine Actions
Baseline	[Global Context]	Script initializes. Allocates memory for globalVar, firstFunction, and secondFunction during creation phase.
Step 2	[firstFunction FEC] [Global Context]	firstFunction() executes. Frame pushed to stack. Scope chain links inward to GEC. Variables inside are scoped locally.
Step 3	[secondFunction FEC] [firstFunction FEC] [Global Context]	Deep execution path triggers secondFunction(). Stack height reaches maximum capacity. Engine focuses resources exclusively here.
Step 4	[firstFunction FEC] [Global Context]	secondFunction reaches termination line. Its memory frame is cleanly unmounted (popped) off the Call Stack.
Final	[Global Context]	

Step	Call Stack Allocation State	Under-the-Hood Engine Actions
		<code>firstFunction</code> completes. Stack pops its frame. Engine returns to tracking base state, waiting for async events.